JavaScript

ВВЕДЕНИЕ

Мы начнём с того, что такое JavaScript, а что нет, и обсудим некоторые способы его использования. Большая часть главы посвящена введению в синтаксис JavaScript — переменные, функции, операторы, циклы и тому подобное. Начнёте ли вы писать код к концу главы? Скорее всего, нет. Но у вас будет хорошее преимущество в понимании того, что происходит в скрипте, когда вы его увидите. Завершу я рассмотрением некоторых способов управления окном браузера и привязки скриптов к действиям пользователя, таким как нажатие кнопки мыши или отправка формы.

ЧТО ТАКОЕ JAVASCRIPT?

JavaScript — это язык программирования, добавляющий интерактивность и пользовательское поведение на наши сайты. Это клиентский язык сценариев, то есть он работает на компьютере пользователя, а не на сервере, как другие языки веб-программирования, такие как PHP и Ruby. Это означает, что JavaScript (и то, как мы его используем) зависит от возможностей и настроек браузера. Он может быть вообще недоступен, либо потому, что пользователь решил отключить его, либо потому, что устройство его не поддерживает, что хорошие разработчики учитывают и учитывают.

JavaScript также известен как динамический язык программирования со слабой типизацией. Не стоит слишком переживать из-за этого описания; позже я объясню, что всё это значит.

Сначала хочу отметить, что JavaScript понимают неправильно.

Чем JavaScript не является

Сразу же название довольно запутанное. Несмотря на своё название, JavaScript не имеет ничего общего с Java. Он был создан Бренданом Айхом в Netscape в 1995 году и изначально назывался «LiveScript». Но в то время Java была на пике популярности, поэтому в маркетинговых целях «LiveScript» превратился в «JavaScript». Или просто «JS», если хотите казаться максимально крутым, говоря о JavaScript.

У JS тоже дурная репутация. Некоторое время он ассоциировался со всевозможными недобросовестными интернет-проделками — нежелательными перенаправлениями, надоедливыми всплывающими окнами и множеством туманных «уязвимостей безопасности» — и это лишь некоторые из них. Было время, когда JavaScript позволял не очень авторитетным разработчикам делать всё это (и даже хуже), но современные браузеры в значительной степени распознали тёмную сторону разработки на JavaScript и заблокировали её.

Однако не стоит винить сам JavaScript за ту эпоху. Как гласит не очень старая поговорка, «с большой силой приходит большая

ответственность». JavaScript всегда предоставлял разработчикам огромный контроль над отображением страниц и поведением наших браузеров, и нам решать, как использовать этот контроль ответственно.

Что это такое

JavaScript — это лёгкий, но невероятно мощный язык сценариев. Чаще всего мы сталкиваемся с ним в наших браузерах, но JavaScript проник повсюду: от нативных приложений до PDF-файлов и электронных книг. Даже сами веб-серверы могут работать на JavaScript. Будучи динамическим языком программирования, JavaScript не нуждается в каком-либо компиляторе, который интерпретирует наш человекочитаемый код в то, что понимает браузер. Браузер фактически считывает код так же, как мы, и интерпретирует его «на лету». JavaScript также слабо типизирован. Всё это означает, что нам не обязательно сообщать JavaScript, что такое переменная. Если мы присваиваем переменной значение 5, нам

не нужно программно указывать эту переменную как число; то есть 5 — это число, и JavaScript распознаёт его именно так.

Что может JavaScript

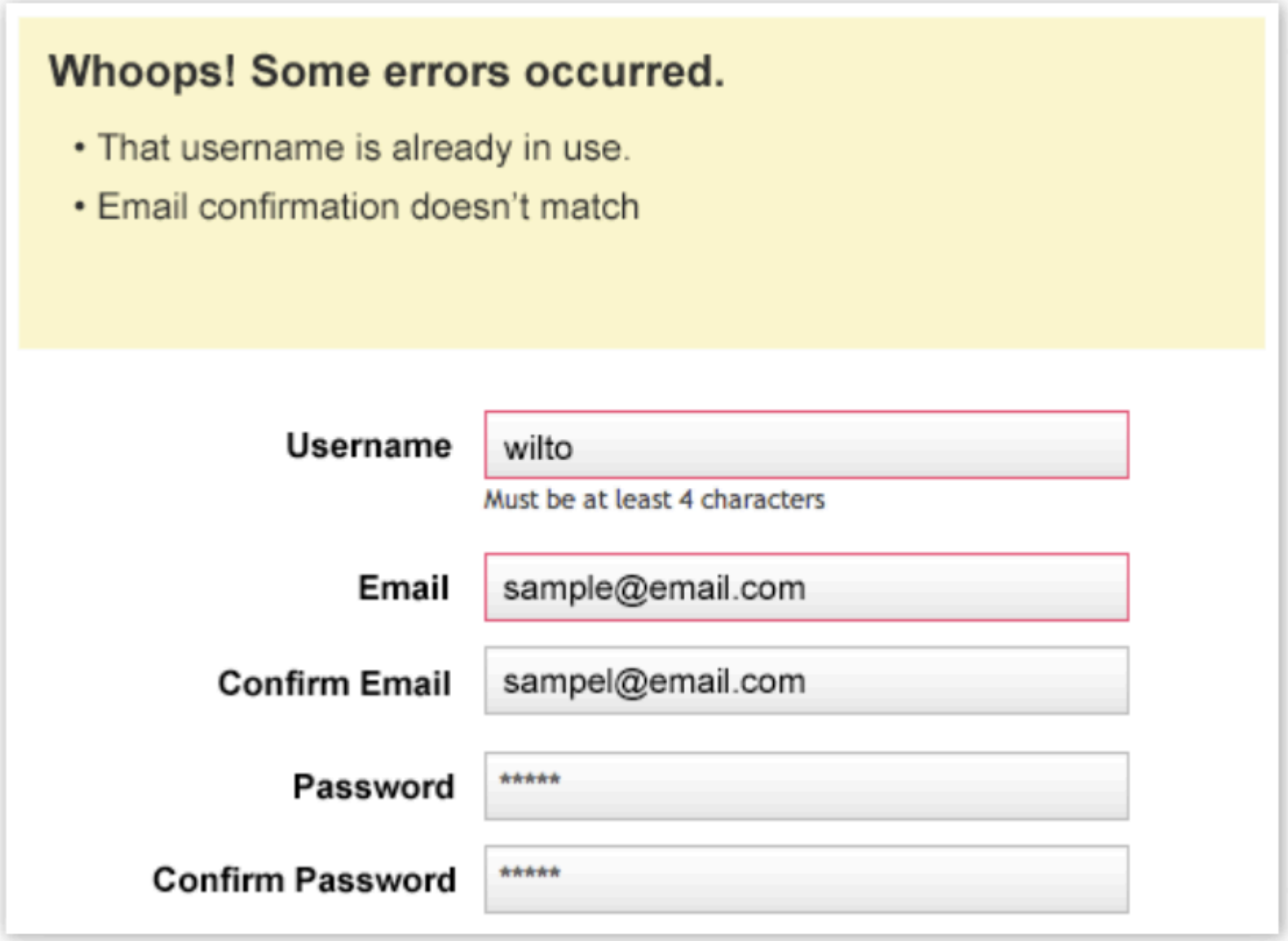
Чаще всего мы сталкиваемся с JavaScript как с способом добавления интерактивности на страницу. В то время как «структурный» уровень страницы — это наша HTML-разметка, а «презентационный» уровень страницы состоит из CSS, третий, «поведенческий» уровень состоит из нашего JavaScript.

Ко всем элементам, атрибутам и тексту на веб-странице могут обращаться скрипты, использующие DOM (объектную модель документа), которую мы рассмотрим позже. Мы также можем писать скрипты, которые реагируют на пользовательский ввод, изменяя содержимое страницы, стили CSS или поведение браузера «на лету».

Вы, вероятно, видели это в действии, если когда-либо пытались зарегистрироваться на сайте, вводили имя пользователя и сразу

получали сообщение о том, что введённое вами имя пользователя уже занято кем-то другим (РИС. 21-1). Красная рамка вокруг поля ввода текста и появление сообщения «Извините, это имя пользователя уже используется» — примеры того, как JavaScript изменяет содержимое страницы.

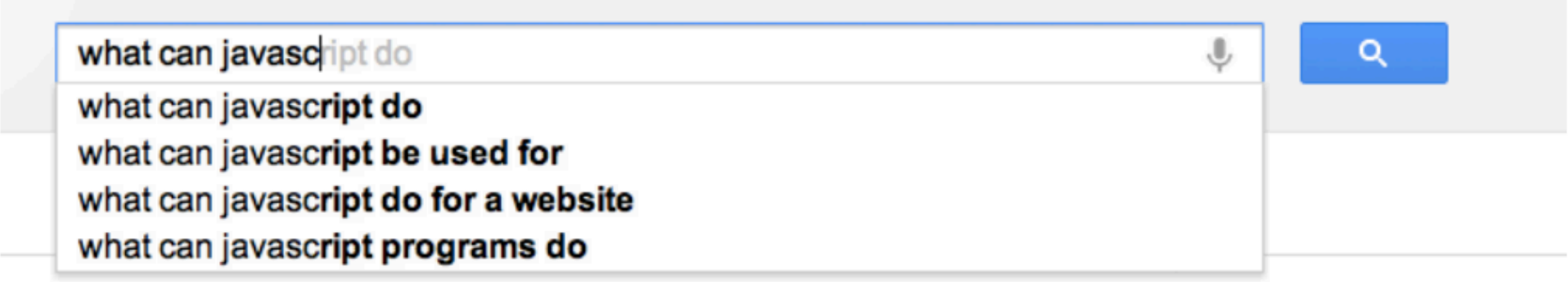
Блокировка отправки формы — пример того, как JavaScript изменяет поведение браузера по умолчанию. В конечном счёте, проверка этой информации — задача сервера, но JavaScript позволяет сайту выполнить этот запрос и предоставить немедленный ответ без необходимости перезагрузки страницы.



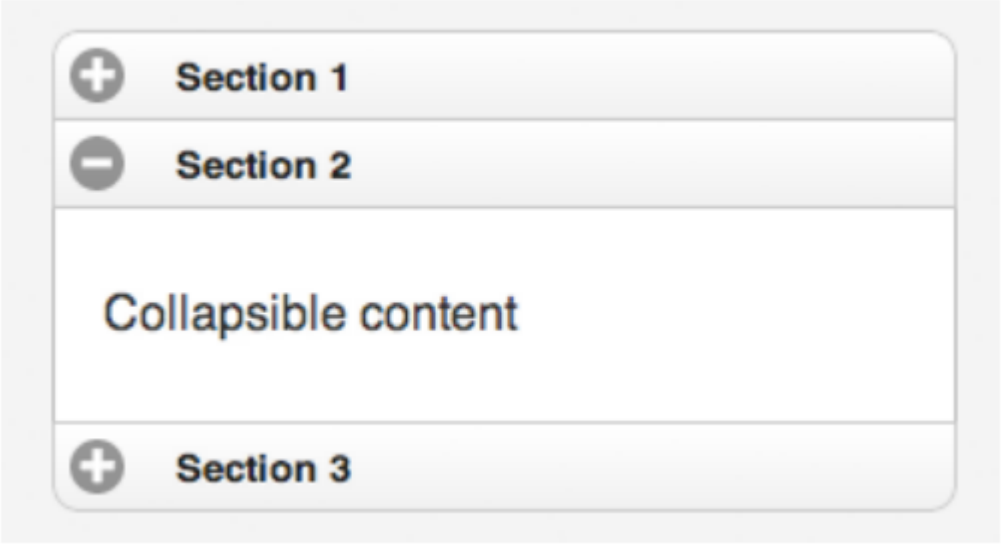
Короче говоря, JavaScript позволяет создавать высокоадаптивные интерфейсы, которые улучшают пользовательский опыт и

обеспечивают динамическую функциональность, не дожидаясь загрузки новой страницы сервером. Например, JavaScript можно использовать для выполнения следующих действий:

Предлагать полный запрос, который пользователь вводит в поле поиска по мере ввода. Вы можете увидеть это на практике на Google.com.



Запрашивать контент и информацию с сервера и внедрять их в текущий документ по мере необходимости, без перезагрузки всей страницы — это обычно называется «Ajax».



Тестировать отдельные функции и возможности браузеров. Например, можно проверить наличие «событий касания», указывающих на то, что пользователь взаимодействует со страницей через браузер мобильного устройства, и добавить больше стилей и методов взаимодействия, оптимизированных для сенсорного экрана.

Заполняют пробелы, в которых встроенная функциональность браузера недостаточна, или добавляют некоторые функции, доступные в новых браузерах, в старые. Такие скрипты обычно называются прокладками или полифиллами.

Загружают изображение или контент в специально стилизованный «лайтбокс», изолированный на странице с помощью CSS, после того, как пользователь нажмет на миниатюру изображения.



```
<div data-role="popup" id="popupBasic">
        This is a completely basic popup, no options set.
</div>
```

This
will f
widgets
Now
Popu
You
the s
by a
have
ther than
erriden
mples

```
<a h                                                                    true"
data
<a h                                                                    rue"
<a href="#transitionExample" data-transition="slideup" data-role="button" data-inline="true
data-rel="popup">Slide up</a>
<a href="#transitionExample" data-transition="slidedown" data-role="button" data-
```

ДОБАВЛЕНИЕ JAVASCRIPT НА СТРАНИЦУ

Как и в случае с CSS, вы можете встроить скрипт прямо в документ или сохранить его во внешнем файле и привязать к странице. В обоих методах используется элемент script.

Встроенный скрипт

Чтобы встроить скрипт на страницу, просто добавьте код в качестве содержимого элемента script:

```
<script>
… Здесь находится код JavaScript
</script>
```

Внешние скрипты

Другой метод использует атрибут src для указания URL-адреса файла скрипта (с суффиксом .js). В этом случае элемент script не

имеет содержимого:

```
<script src="my_script.js"></script>
```

Преимущество внешних скриптов заключается в том, что один и тот же скрипт можно применять к нескольким страницам (то же преимущество, что и внешние таблицы стилей). Недостатком, конечно, является то, что каждый внешний скрипт требует дополнительного HTTP-запроса к серверу, что снижает производительность.

Размещение скрипта

Элемент script может располагаться в любом месте документа, но чаще всего скрипты располагаются в заголовке и в самом конце основного текста. Рекомендуется не разбрасывать их по всему документу, так как в этом случае их будет сложно найти и поддерживать.

Для большинства скриптов предпочтительным местом является конец документа, непосредственно перед тегом `</body>`, поскольку браузер завершит анализ документа и его DOM-структуры:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
</head>
<body>
…содержимое страницы…
<script src="script.js"></script>
</body>
</html>
```

Следовательно, эта информация будет готова и доступна к моменту её поступления в скрипты, и они смогут выполняться быстрее. Кроме того, загрузка и выполнение скрипта блокируют рендеринг

страницы, поэтому перемещение скрипта в конец страницы улучшает воспринимаемую производительность.

Однако в некоторых случаях может потребоваться, чтобы скрипт выполнил какие-либо действия до полной загрузки тела страницы, поэтому размещение его в заголовке страницы повысит производительность. Например, Modernizr (инструмент для обнаружения функций) рекомендует размещать свой скрипт в разделе head, чтобы тесты на обнаружение функций можно было запустить заранее.

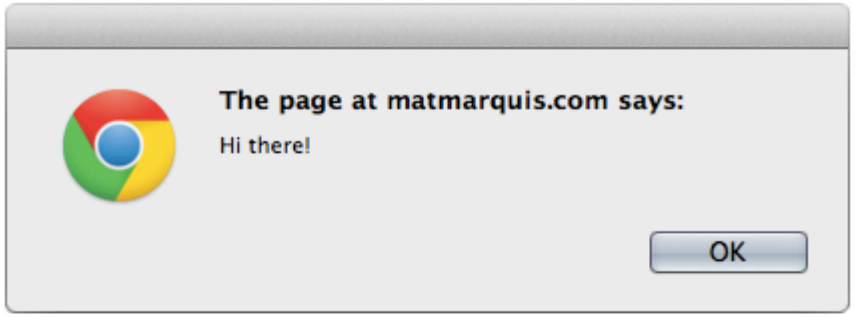
АНАТОМИЯ СКРИПТА

В этом разделе мы познакомим вас с основными строительными блоками JavaScript, чтобы вы могли начать понимать скрипты, когда с ними столкнётесь. Многие разработчики научились программировать, находя существующие скрипты и адаптируя их под свои нужды. После некоторой практики они готовы начать писать

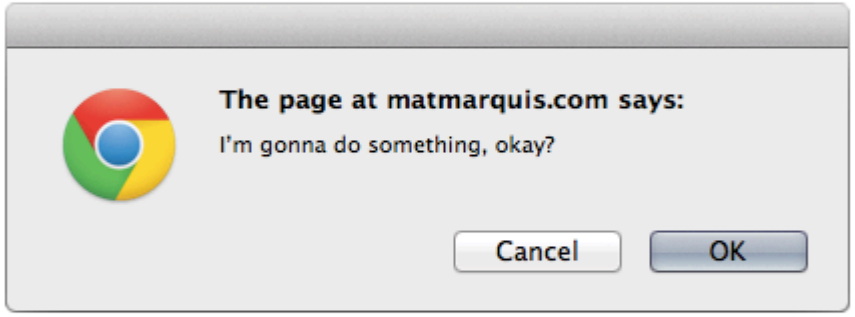
свои собственные с нуля. Распознавание частей скрипта — это первый шаг, поэтому с этого мы и начнём.

Изначально функциональность JavaScript в основном ограничивалась примитивными методами взаимодействия с пользователем. Мы могли использовать несколько встроенных функций JavaScript (РИС. 21-5) для обеспечения обратной связи с пользователем, например, alert() для отправки уведомления пользователю и confirm() для запроса подтверждения или отклонения действия. Для запроса ввода данных пользователя мы были более или менее ограничены встроенной функцией prompt(). Хотя эти методы всё ещё актуальны и сегодня, они раздражают, навязчивы и — по крайней мере, по общему мнению — довольно неприятны в плане взаимодействия с пользователями. С развитием JavaScript у нас появились гораздо более изящные способы добавления поведения на наши страницы, что делает их использование более удобным для пользователей.

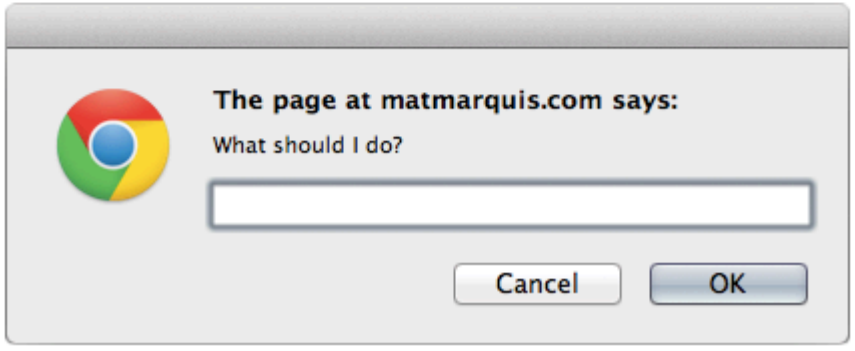
```
alert("Hi there");
```




```
confirm("I'm gonna do something, okay?");
```




```
prompt("What should I do?");
```



Чтобы воспользоваться этими методами взаимодействия, необходимо сначала понять логику, лежащую в основе скриптов. Это логические шаблоны, общие для всех языков программирования, хотя синтаксис может различаться.

Проведем параллель между языками программирования и разговорными языками: хотя словарный запас может различаться от языка к языку, многие грамматические шаблоны являются общими для большинства из них. К концу этого раздела вы узнаете о переменных, массивах, операторах сравнения, операторах if/else, циклах, функциях и многом другом.

Основы

Существует несколько общих синтаксических правил, которые пронизывают весь JavaScript.

Важно знать, что JavaScript чувствителен к регистру. Переменная с именем myVariable, переменная с именем myvariable и переменная с именем MYVariable будут рассматриваться как три разных объекта.

Кроме того, пробелы, такие как табуляция и пробелы, игнорируются, если только они не являются частью текстовой строки и не заключены в кавычки. Все пробелы, добавленные в скрипты, подобные тем, что представлены в этой главе, предназначены для удобства чтения человеком — они упрощают чтение кода. JavaScript их не видит.

Операторы

Скрипт состоит из последовательности операторов. Оператор — это команда, сообщающая браузеру, что делать. Вот простой оператор, который заставляет браузер вывести предупреждение с фразой «Спасибо»:

```
alert("Спасибо.");
```

Точка с запятой в конце оператора сообщает JavaScript, что это конец команды, так же как точка завершает предложение. Согласно стандарту JavaScript, перенос строки также означает конец

команды, но рекомендуется заканчивать каждый оператор точкой с запятой.

Комментарии

JavaScript позволяет оставлять комментарии, которые будут проигнорированы во время выполнения скрипта, что позволяет добавлять напоминания и пояснения по всему коду. Это особенно полезно, если этот код, вероятно, будет редактироваться другим разработчиком в будущем.

Существует два способа использования комментариев. Для однострочных комментариев используйте два символа косой черты (//) в начале строки. Однострочные комментарии можно размещать на той же строке, что и оператор, при условии, что комментарий следует после оператора. Закрывать комментарий не обязательно, так как перенос строки фактически закрывает его.

```
// Это однострочный комментарий.
```

Многострочные комментарии используют тот же синтаксис, что и в CSS. Всё, что находится между символами /* */, игнорируется браузером. Вы можете использовать этот синтаксис для «комментирования» заметок и даже фрагментов скрипта при устранении неполадок.

```
/* Это многострочный комментарий.
Всё, что находится между этими наборами символов, будет
полностью проигнорировано при выполнении скрипта.
Этот комментарий необходимо закрыть. */
```

Переменные

Переменная подобна контейнеру информации. Вы даёте ей имя, а затем присваиваете значение, которое может быть числом, текстовой строкой, элементом DOM или функцией — да чем угодно. Это даёт нам удобный способ впоследствии ссылаться на это значение по имени. Само значение можно изменять и переназначать любым способом, который диктует логика наших скриптов.

Следующее объявление создаёт переменную с именем foo и присваивает ей значение 5:

```
var foo = 5;
```

Мы начинаем с объявления переменной с помощью ключевого слова var. Одиночный знак равенства (=) означает, что мы присваиваем ей значение. Поскольку это конец нашего оператора, мы завершаем строку точкой с запятой. Переменные также можно объявлять без ключевого слова var, что влияет на то, какая часть вашего скрипта будет иметь доступ к содержащейся в них информации. Мы обсудим это далее в разделе «Область действия переменной и ключевое слово var».

Вы можете использовать любое имя в качестве имени переменной, но убедитесь, что оно будет понятно вам в дальнейшем. Не стоит называть переменную, например, data; имя должно описывать содержащуюся в ней информацию. В нашем предыдущем конкретном

примере productName может быть более подходящим именем, чем foo. Существует несколько правил именования переменных:

Имя должно начинаться с буквы или символа подчёркивания.

Имя может содержать буквы, цифры и символы подчёркивания в любых сочетаниях.

Имя не должно содержать пробелов. В качестве альтернативы используйте подчёркивания вместо пробелов или закрывайте пробел и используйте «верблюжий регистр» (например, my_variable или myVariable).

Имя не должно содержать специальных символов (например, ! . , / \ + * =).

Вы можете изменить значение переменной в любой момент, повторно объявив её в любом месте скрипта. Помните: JavaScript чувствителен к регистру, как и имена переменных.

Типы данных

Значения, которые мы присваиваем переменным, относятся к нескольким различным типам данных:

Undefined

Простейшим из этих типов данных, скорее всего, является undefined. Если мы объявляем переменную, давая ей имя, но не давая значения, эта переменная будет содержать значение undefined.

```
var foo;
alert(foo); // This will open a dialog containing "undefined".
```

Скорее всего, вы не сразу найдёте этому применение, но об этом стоит знать, чтобы избежать ошибок, с которыми вы, вероятно, столкнётесь на ранних этапах своей карьеры в JavaScript. Если переменная имеет значение undefined, хотя это не так, стоит ещё раз проверить, правильно ли она объявлена или нет ли опечатки в её имени.

Null

Аналогично undefined, присвоение переменной значения null (опять же, с учётом регистра) просто означает: «Определите эту переменную, но не присваивайте ей собственного значения».

```
var foo = null;
alert(foo); // Откроется диалоговое окно, содержащее «null».
```

Числа

Вы можете присваивать переменным числовые значения.

```
var foo = 5;
alert(foo); // Откроется диалоговое окно, содержащее «5».
```

Слово foo теперь означает в JavaScript то же самое, что и число 5. Поскольку JavaScript слабо типизирован, нам не нужно указывать

скрипту, что переменная foo должна обрабатываться как число 5. Переменная ведёт себя так же, как и само число, поэтому с ней можно делать то же, что и с любым другим числом, используя классические математические обозначения: +, -, * и / для сложения, вычитания, умножения и деления соответственно. В этом примере мы используем знак плюс (+), чтобы сложить foo с самим собой (foo + foo).

```
var foo = 5;
alert(foo + foo); // This will alert "10".
```

Строки

Другой тип данных, который можно сохранить в переменной, — это строка, которая, по сути, представляет собой строку текста. Заключение символов в одинарные или двойные кавычки указывает на то, что это строка, как показано здесь:

```
var foo = "five";
alert( foo ); // Это выведет оповещение о "five"
```

Переменная foo теперь обрабатывается точно так же, как и слово "five". Это применимо к любой комбинации символов: букв, цифр, пробелов и так далее. Если значение заключено в кавычки, оно будет обрабатываться как текстовая строка. Если мы заключим число 5 в кавычки и присвоим его переменной, эта переменная не будет вести себя как число; вместо этого она будет вести себя как текстовая строка, содержащая символ "5". Строки

Ранее мы видели знак плюс (+), используемый для сложения чисел. При использовании знака «плюс» со строками строки объединяются (конкатенация) в одну длинную строку, как показано в этом примере.

```
var foo = "bye"
alert(foo + foo); // Это выведет оповещение "byebye"
```

Обратите внимание, что возвращает оповещение в следующем примере, когда мы определяем значение 5 в кавычках, рассматривая его как строку, а не как число:

```
var foo = "5";
alert( foo + foo ); // Это выведет оповещение "55"
```

Если мы объединим строку и число, JavaScript предположит, что число также следует рассматривать как строку, поскольку математические вычисления будут невозможны.

```
var foo = "five";
var bar = 5;
alert( foo + bar ); // Это выведет предупреждение "five5"
```

Логические значения

Мы также можем присвоить переменной значение true или false. Это называется логическим значением и является основой для всех

видов сложной логики. Логические значения используют встроенные в JavaScript ключевые слова true и false, поэтому кавычки не нужны.

```
var foo = true; // Переменная "foo" теперь имеет значение true
```

Как и в случае с числами, если бы мы заключили предыдущее значение в кавычки, мы бы сохранили слово true в нашей переменной вместо изначального значения true (т. е. «не false»).

В некотором смысле, всё в JavaScript имеет либо изначально истинное, либо изначально ложное значение. Например, null, undefined, 0 и пустые строки ("") изначально ложны, в то время как любое другое значение изначально ложно. Эти значения, хотя и не идентичны логическим значениям true и false, обычно называются «истинными» и «ложными».

Массивы

Массив — это группа нескольких значений (называемых элементами), которые можно присвоить одной переменной. Значения в массиве называются индексированными, то есть к ним можно обращаться по номеру в соответствии с порядком их расположения в списке. Первому элементу присваивается индекс 0, второму — 1 и так далее. Именно поэтому мы, зануды, почти всегда начинаем считать с нуля — ведь именно так считает JavaScript, и многие другие языки программирования делают то же самое. Помня об этом, мы можем избежать множества проблем с кодированием в будущем.

Итак, предположим, что нашему скрипту нужны все переменные, которые мы определили ранее. Мы могли бы определить их три раза и назвать их как-то вроде foo1, foo2 и так далее, или сохранить их в массиве, обозначенном квадратными скобками ([ ]).

```
var foo = [5, "five", "5"];
```

Теперь, когда вам понадобится доступ к любому из этих значений, вы можете получить их из единого массива foo, указав их индекс:

```
alert( foo[0] ); // Оповещения "5"
alert( foo[1] ); // Оповещения "5"
alert( foo[2] ); // Также оповещения "5"
alert( foo[0] ); // Оповещения "5"
alert( foo[1] ); // Оповещения "5"
alert( foo[2] ); // Также выдаёт "5"
```

Операторы сравнения

Теперь, когда мы знаем, как сохранять значения в переменных и массивах, следующим логическим шагом станет изучение того, как сравнивать эти значения. Существует набор специальных символов, называемых операторами сравнения, которые оценивают и сравнивают значения различными способами:

`==` Равно

`!=` Не равно

`===` Идентичен (равно и относится к тому же типу данных)

`!==` Не тождественен

`>` Больше

`>=` Больше или равно

`<` Меньше

`<=` Меньше или равно

Недаром все эти определения воспринимаются как части утверждения. Сравнивая значения, мы делаем утверждение, и цель — получить результат, который либо изначально истинен, либо изначально ложен. Когда мы сравниваем два значения, JavaScript оценивает утверждение и возвращает логическое значение в зависимости от того, истинно оно или ложно.

```
alert( 5 == 5 ); // Это выдаст предупреждение «истина»
alert( 5 != 6 ); // Это выдаст предупреждение «истина»
alert( 5 < 1 ); // Это выдаст предупреждение «ложь»
```

Равно и идентично

Сложность заключается в понимании разницы между «равно» (==) и «идентично» (===). Мы уже знаем, что все эти значения относятся к определённому типу данных. Например, строка из «5» и число 5 похожи, но это не совсем одно и то же.

Что ж, именно для этого и предназначен оператор ===.

```
alert( "5" == 5 ); // Это выведет предупреждение "true". Оба значения "5".
alert( "5" === 5 ); /* Это выведет предупреждение "false". Оба значения "5", но это разные типы данных. */
alert( "5" !== 5 ); /* Это выведет предупреждение "true", поскольку это разные типы данных. */
```

Математические операторы

Другой тип операторов — это математические операторы, которые выполняют математические операции с числовыми значениями (и, конечно же, переменными, содержащими числовые значения). Мы кратко коснулись простых математических операторов для сложения

(+), вычитания (-), умножения (*) и деления (/). Есть также несколько полезных сокращений, о которых вам следует знать:

`+=` Складывает значение с самим собой
`++` Увеличивает значение числа (или переменной, содержащей числовое значение) на 1
`--` Уменьшает значение числа (или переменной, содержащей числовое значение) на 1

Операторы if/else

Благодаря операторам if/else JavaScript задаёт себе вопрос «истина/ложь». Они, по сути, являются основой всей сложной логики, которую можно реализовать на JavaScript, и они настолько просты, насколько это вообще возможно в программировании. Фактически, они написаны практически на обычном английском языке. Структура условного оператора выглядит следующим образом:

```
if( true ) {
// Выполнить что-то.
}
```

Он сообщает браузеру: «Если это условие выполнено, выполнить команды, перечисленные в фигурных скобках ({ })». Помните, JavaScript не заботится о пробелах в нашем коде, поэтому пробелы по обе стороны от (true) нужны исключительно для большей читабельности кода.

Вот простой пример использования массива, который мы объявили ранее:

```
var foo = [5, "five", "5"];
if( foo[1] === "five" ) {
alert("Это слово "five", написанное обычным языком.");
}
```

Поскольку мы выполняем сравнение, JavaScript вернет нам значение true или false. Выделенная строка кода говорит: «true или false: значение переменной foo с индексом 1 идентично слову „five“?»

В этом случае оповещение сработает, потому что переменная foo с индексом 1 (вторая в списке, если вы помните) идентична числу «пять». Это действительно так, и оповещение сработает.

Мы также можем явно проверить значение на ложность, используя оператор сравнения !=, который читается как «не равно».

```
if( 1 != 2 ) {
alert("Если вы не видите это оповещение, у нас проблемы поважнее, чем в JavaScript."); // 1 никогда не равно 2, поэтому мы всегда должны видеть это оповещение.
}
```

JavaScript говорит: «Эта строка «1 не равно 2» — истинное утверждение, поэтому я выполню этот код».

Если выражение не истинно, код в фигурных скобках будет полностью пропущен:

```
if( 1 == 2 ) {
alert("Если вы видите это предупреждение, у нас проблемы поважнее, чем JavaScript."); // 1 не равно 2, поэтому этот код никогда не выполнится.
}
```

Это касается «if», но как насчёт «else»?

Что, если мы хотим сделать одно действие, если что-то истинно, и что-то другое, если это ложно? Мы могли бы написать два оператора if, но это будет немного неуклюже. Вместо этого мы можем просто сказать: «иначе, сделай что-нибудь…иначе».

```
var test = "testing";
if( test == "testing" ) {
alert( "Вы ничего не изменили." );
} else {
```

```
alert( "Вы что-то изменили!" );
}
```

Изменение значения тестовой переменной на что-либо другое (кроме слова "testing") вызовет оповещение «Вы что-то изменили!»

Циклы

Бывают случаи, когда нам нужно перебрать каждый элемент массива и что-то с ним сделать, но при этом не хочется выписывать весь список элементов и повторять это дюжину или более раз. Циклы невероятно полезны.

С учётом того, что мы уже рассмотрели, мы неплохо учимся работать с отдельными переменными, но этого недостаточно. Циклы позволяют нам легко работать с большими наборами данных.

Допустим, у нас есть форма, в которой ни одно поле не должно быть пустым. Форма представлена в виде массива, поэтому мы могли бы проверять каждое значение, хранящееся в этом массиве,

по одному элементу за раз, но это потребовало бы много кода и было бы кошмаром в обслуживании. Если мы будем использовать цикл для проверки каждого значения, нам не придётся изменять наш скрипт, независимо от того, сколько полей добавляется или удаляется на странице. Циклы позволяют нам обрабатывать каждый элемент массива, независимо от его размера.

Существует несколько способов написать цикл, но метод for — один из самых популярных. Базовая структура цикла for выглядит следующим образом:

```
for( инициализация переменной; проверка условия; изменение значения;
) {
// выполнение какого-либо действия
}
```

Вот пример цикла for в действии:

```
for( var i = 0; i < 2; i++ ) {
alert( i ); // Этот цикл вызовет три оповещения: «0», «1» и «2»
соответственно.
}
```

Слишком много, поэтому давайте разберёмся:

for()

Сначала мы вызываем оператор for(), встроенный в JavaScript. Он говорит: «Каждый раз, когда это истинно, выполнить это». Затем нам нужно предоставить этому оператору некоторую информацию.

var i = 0;

Это создаёт новую переменную i со значением, равным нулю. Одинарный знак равенства указывает на то, что это переменная. Чаще всего вы увидите, как программисты используют букву «i» (сокращение от «index») в качестве имени переменной, но имейте в виду, что вместо неё можно использовать любое имя переменной. Это общепринятое соглашение, а не правило.

Мы устанавливаем это начальное значение равным 0, чтобы сохранить привычку считать от нуля. В конце концов, JavaScript именно с этого и начинает отсчёт.

i <= 2;

С i <= 2; мы говорим: «Пока i меньше или равно 2, цикл продолжается». Поскольку мы считаем с нуля, это означает, что цикл будет выполнен три раза.

i++

Наконец, i++ — это сокращение для «каждый раз, когда этот цикл выполняется, прибавлять единицу к значению i» (++ — один из математических сокращённых операторов, которые мы рассматривали ранее). Без этого шага i всегда было бы равно нулю, и цикл выполнялся бы вечно! К счастью, современные браузеры достаточно умны, чтобы не допустить этого. Если один из этих трёх элементов отсутствует, цикл просто не запустится.

{ script }

Всё, что находится внутри фигурных скобок, выполняется один раз

за каждый запуск цикла, то есть три раза в данном случае. Переменная i также доступна для использования в коде, который выполняется циклом, как мы увидим далее.

Вернёмся к примеру «проверка каждого элемента массива». Как написать цикл, который делает это за нас?

```
var items = ["foo", "bar", "baz"]; // Сначала создаём массив.
for( var i = 0; i < items.length; i++ ) {
alert( items[i] ); // Это выведет оповещение на каждый элемент
массива.
}
```

Этот пример отличается от нашего первого цикла двумя ключевыми моментами:

items.length

Вместо использования числа для ограничения количества повторений цикла мы используем свойство, встроенное в JavaScript, для определения «длины» нашего массива, то есть количества

содержащихся в нём элементов. .length — это лишь одно из стандартных свойств и методов объекта Array в JavaScript. В нашем примере массив содержит три элемента, поэтому цикл будет выполнен три раза.

items[i]

Помните, как я упоминал, что мы можем использовать переменную i внутри цикла? Что ж, мы можем использовать её для ссылки на каждый индекс массива. Хорошо, что мы начали считать с нуля; если бы мы установили начальное значение i равным 1, первый элемент массива был бы пропущен. Результатом нашего примера цикла for является то, что каждый элемент массива (текстовые строки foo, bar и baz) возвращается после каждого цикла и передается в оповещение.

Теперь, независимо от того, насколько большим или маленьким должен быть этот массив, цикл будет выполняться ровно столько раз, сколько элементов в массиве, и всегда будет содержать удобную ссылку на каждый элемент массива.

Существуют буквально десятки способов написать цикл в JavaScript, но это один из наиболее распространённых шаблонов, с которым вы столкнётесь в реальной жизни. Разработчики используют циклы для выполнения ряда задач, например:

Циклический проход по списку элементов на странице и проверка значения каждого элемента, применение стиля к каждому элементу или добавление/удаление/изменение атрибута каждого элемента. Например, мы можем пройти циклом по каждому элементу формы и убедиться, что пользователь ввёл для каждого элемента допустимое значение, прежде чем продолжить.

Создание нового массива элементов исходного массива с определённым значением. Мы проверяем значение каждого элемента исходного массива в цикле, и если значение совпадает с искомым, мы заполняем новый массив только этими элементами. Это превращает цикл в своего рода фильтр.

Функции

Я уже познакомил вас с несколькими функциями. Вот пример функции, которая, возможно, вам знакома:

```
alert("Я всё это время был функцией!");
```

Функция — это фрагмент кода для выполнения задачи, которая не выполняется, пока на неё не ссылаются или не вызывают. alert() — это встроенная в наш браузер функция. Это блок кода, который запускается только тогда, когда мы явно ему это указываем. В некотором смысле функцию можно представить как переменную, содержащую логику, то есть обращение к этой переменной приведёт к запуску всего хранящегося в ней кода. Функции позволяют повторно использовать код при каждом обращении к нему, поэтому вам не нужно писать его снова и снова.

Все функции имеют общий шаблон (РИС. 21-6). За именем функции всегда следуют скобки (без пробелов), а затем пара фигурных скобок, содержащая соответствующий код. Скобки иногда содержат дополнительную информацию, используемую функцией, называемую

аргументами. Аргументы — это данные, которые могут влиять на поведение функции. Например, хорошо знакомая нам функция alert() принимает строку текста в качестве аргумента и использует эту информацию для заполнения результирующего диалогового окна.

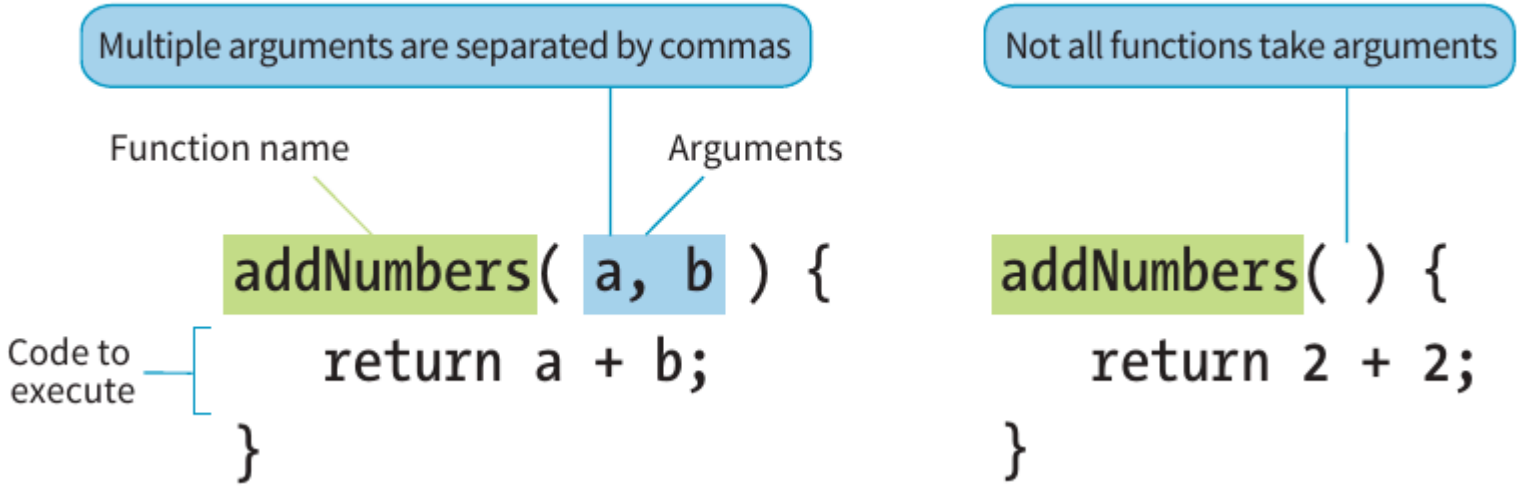


Рисунок 6

Существует два типа функций: изначально встроенные (нативные функции JavaScript) и создаваемые вами самостоятельно (пользовательские функции). Давайте рассмотрим каждый из них.

Встроенные функции

В JavaScript встроены сотни предопределённых функций, включая следующие:

```
alert(), confirm() и prompt()
```

Эти функции вызывают диалоговые окна на уровне браузера.

```
Date()
```

Возвращает текущую дату и время.

```
parseInt("123")
```

Эта функция, помимо прочего, принимает строковый тип данных, содержащий числа, и преобразует его в числовой тип данных. Строка передаётся функции в качестве аргумента.

```
setTimeout(functionName, 5000)
```

Выполняет функцию с задержкой. Функция указана в первом аргументе, а задержка в миллисекундах — во втором (в примере 5000 миллисекунд, что равно 5 секундам). Существуют и другие значения. Обратите внимание, что имена функций чувствительны к регистру, поэтому обязательно используйте setTimeout вместо SetTimeout.

Пользовательские функции

Чтобы создать пользовательскую функцию, мы вводим ключевое слово function, затем имя функции, открывающую и закрывающую скобки, а затем открывающую и закрывающую фигурные скобки:

```
function name() {
// Здесь будет код нашей функции.
}
```

Как и в случае с переменными и массивами, имя функции может быть любым, но правила синтаксиса именования остаются теми же. Если бы мы создали функцию, которая просто выводит текстовое оповещение, она бы выглядела так:

```
function foo() {
alert("Наша функция только что выполнилась!"); // Этот код не выполнится, пока мы не вызовем функцию 'foo()'
}
```

Затем мы можем вызвать эту функцию и выполнить код внутри неё в любом месте нашего скрипта, написав следующее:

```
foo(); // Оповещение "Наша функция только что выполнилась!"
```

Мы можем вызывать эту функцию любое количество раз в нашем коде. Это экономит много времени и избавляет от лишнего кодирования.

Аргументы

Наличие функции, которая выполняет один и тот же код во всем скрипте, вряд ли будет полезным. Мы можем «передавать аргументы» (предоставлять данные) встроенным и пользовательским функциям, чтобы применять логику функции к разным наборам данных в разное время. Чтобы выделить место для аргументов, создайте имя переменной (или последовательность имён, разделённых запятыми) в скобках после имени функции во время её определения.

Например, мы хотим создать очень простую функцию, которая выводит количество элементов в массиве. Мы уже знаем, что можно использовать .length для получения количества элементов в массиве, поэтому нам просто нужно передать измеряемый массив в нашу функцию. Мы делаем это, передавая измеряемый массив в качестве аргумента. В коде я определил новую функцию alertArraySize() и создал переменную arr, которая хранит место для

аргумента. Эта переменная будет доступна внутри функции и будет содержать любой аргумент, переданный нами при вызове функции.

```
function alertArraySize(arr) {
alert(arr.length);
}
```

При вызове этой функции всё, что мы добавим в скобки после имени функции (в данном случае test), будет передано аргументу с заполнителем arr во время выполнения функции. Здесь мы определили переменную test как массив из пяти элементов. Мы передали эту переменную в функцию, и теперь этот массив подключается, и возвращается длина.

```
var test = [1,2,3,4,5];
alertArraySize(test); // Оповещения "5"
```

Возврат значения

Очень часто функция используется для вычисления чего-либо, а затем возвращает значение, которое можно использовать в другом месте скрипта. Мы могли бы добиться этого, используя то, что нам известно сейчас, посредством грамотного применения переменных, но есть гораздо более простой способ.

Ключевое слово return внутри функции фактически превращает эту функцию в переменную с динамическим значением! Это немного проще продемонстрировать, чем объяснить, поэтому наберитесь терпения, пока мы рассмотрим следующий пример:

```
function addNumbers(a,b) {
return a + b;
}
```

Теперь у нас есть функция, которая принимает два аргумента и складывает их. Это было бы бесполезно, если бы результат всегда находился внутри функции, потому что мы не смогли бы использовать его где-либо ещё в нашем скрипте. Здесь мы

используем ключевое слово return для передачи результата из функции. Теперь любая ссылка на эту функцию возвращает результат функции — как переменная:

```
alert( addNumbers(2,5) ); // Оповещения "7"
```

В некотором смысле, функция addNumbers() теперь является переменной, содержащей динамическое значение: значение нашего вычисления. Если бы мы не вернули значение внутри нашей функции, предыдущий скрипт выдал бы оповещение undefined, как и переменная, которой не было присвоено значение.

У ключевого слова return есть одна загвоздка. Как только JavaScript обнаруживает, что пришло время вернуть значение, функция завершается. Рассмотрим следующий пример:

```
function bar() {
return 3;
```

```
alert("Мы никогда не увидим это оповещение.");
}
```

При вызове этой функции с помощью bar() оповещение во второй строке никогда не срабатывает. Функция завершает работу, как только обнаруживает, что пришло время вернуть значение.

Область действия переменной и ключевое слово var

Бывают случаи, когда требуется, чтобы переменная, определённая внутри функции, была доступна в любом месте скрипта. В других случаях может потребоваться ограничить её и сделать доступной только для функции, в которой она находится. Эта доступность переменной называется её областью действия. Переменная, которая может использоваться любым скриптом на странице, имеет глобальную область действия, а переменная, доступная только внутри своей родительской функции, имеет локальную область действия.

Переменные JavaScript используют функции для управления своей областью действия. Если переменная определена вне функции, она будет иметь глобальную область действия и будет доступна всем скриптам. Если вы определяете переменную внутри функции и хотите, чтобы она использовалась только этой функцией, вы можете отметить её как локальную область видимости, добавив перед именем переменной ключевое слово var:

```
var foo = "value";
```

Чтобы сделать переменную внутри функции глобальной, мы опускаем ключевое слово var и просто определяем переменную:

```
foo = "value";
```

Вам следует быть внимательным при определении переменных внутри функций, иначе вы можете получить неожиданные результаты. Рассмотрим, например, следующий фрагмент JavaScript:

```
function double(num){
total = num + num;
return total;
}
var total = 10;
var number = double(20);
alert(total); // Оповещения 40.
```

Можно ожидать, что, поскольку вы специально присвоили переменной total значение 10, функция alert(total) в конце скрипта вернет 10. Но поскольку мы не обозначили переменную total в функции ключевым словом var, она попадает в глобальную область видимости. Поэтому, хотя переменная total установлена равной 10, следующий оператор запускает функцию и извлекает значение total, определённое там. Без var переменная «вытекала».

Как видите, проблема с глобальными переменными заключается в том, что они будут использоваться совместно во всех скриптах на странице. Чем больше переменных попадает в глобальную область

видимости, тем выше вероятность столкнуться с «коллизией», когда переменная, названная где-то ещё (даже в другом скрипте), совпадает с одной из ваших переменных. Это может привести к непреднамеренному переопределению переменных с неожиданными значениями, что может привести к ошибкам в вашем скрипте.

SCOPE CHEAT SHEET

Variable	Location	Scope
var identifier value	Outside a function	Global
var identifier value	Inside a function	Local
identifier value	Inside a function	Global

Помните, что мы не всегда можем контролировать весь код, используемый на нашей странице. Очень часто страницы содержат код, написанный сторонними разработчиками, например:

Скрипты для отображения рекламы

Скрипты для отслеживания пользователей и аналитики

Кнопки «Поделиться» в социальных сетях

Лучше не рисковать конфликтами переменных, поэтому, когда вы начинаете писать скрипты самостоятельно, по возможности устанавливайте локальную область видимости для своих переменных.

На этом завершается наш небольшой вводный обзор синтаксиса JavaScript. Он гораздо сложнее, но это должно дать вам хорошую основу для самостоятельного изучения и умения интерпретировать скрипты. Нам осталось рассмотреть ещё несколько особенностей JavaScript, прежде чем мы рассмотрим несколько примеров.

ОБЪЕКТ БРАУЗЕРА

Помимо управления элементами веб-страницы, JavaScript также предоставляет доступ к частям самого окна браузера и возможность манипулировать ими. Например, вам может понадобиться получить

или заменить URL-адрес в адресной строке браузера, а также открыть или закрыть окно браузера.

В JavaScript браузер называется объектом окна (window). У объекта окна есть ряд свойств и методов, которые мы можем использовать для взаимодействия с ним. Фактически, наш старый знакомый alert() — это один из стандартных методов объекта браузера. В ТАБЛИЦЕ 1 перечислены лишь некоторые свойства и методы, которые можно использовать с window, чтобы дать вам представление о возможностях. Полный список см. в справочнике Window API на сайте MDN Web Docs (developer.mozilla.org/en-US/docs/Web/API/Window).

Property/method	Description
event	Represents the state of an event
history	Contains the URLs the user has visited within a browser window
location	Gives read/write access to the URI in the address bar
status	Sets or returns the text in the status bar of the window
alert()	Displays an alert box with a specified message and an OK button
close()	Closes the current window
confirm()	Displays a dialog box with a specified message and an OK and a Cancel button
focus()	Sets focus on the current window

Таблица 1

СОБЫТИЯ

JavaScript может получать доступ к объектам на странице и в окне браузера, но знаете ли вы, что он также «ожидает» определённых событий? Событие — это действие, которое можно обнаружить с

помощью JavaScript, например, загрузка документа, щелчок пользователя по элементу или просто наведение на него указателя мыши. HTML 4.0 позволил привязывать скрипт к событиям на странице, инициированным пользователем, самим браузером или другими скриптами. Это называется привязкой событий.

В скриптах событие идентифицируется обработчиком событий. Например, обработчик событий onload запускает скрипт при загрузке документа, а обработчики onclick и onmouseover запускают скрипт, когда пользователь щёлкает мышью по элементу или наводит указатель мыши на него соответственно. В ТАБЛИЦЕ 21-2 перечислены некоторые из наиболее распространённых обработчиков событий. Имейте в виду, что они также чувствительны к регистру.

Event handler	Event description
onblur	An element loses focus.
onchange	The content of a form field changes.
onclick	The mouse clicks an object.
onerror	An error occurs when the document or an image loads.
onfocus	An element gets focus.
onkeydown	A key on the keyboard is pressed.
onkeypress	A key on the keyboard is pressed or held down.
onkeyup	A key on the keyboard is released.
onload	A page or an image is finished loading.
onmousedown	A mouse button is pressed.
onmousemove	The mouse is moved.
onmouseout	The mouse is moved off an element.
onmouseover	The mouse is moved over an element.
onmouseup	A mouse button is released.
onsubmit	The submit button is clicked in a form.

Таблица 2

Существует три распространённых способа применения обработчиков событий к элементам на наших страницах:

Как HTML-атрибут
Как метод, прикреплённый к элементу
Использование addEventListener()

В следующих примерах последних двух подходов мы будем использовать объект window. Любые события, прикреплённые к window, применяются ко всему документу. Во всех этих случаях мы также будем использовать событие onclick.

Как атрибут HTML

Вы можете указать функцию, которая будет запущена, в атрибуте разметки, как показано в следующем примере:

```
<body onclick="myFunction();"> /* myFunction теперь будет запускаться при щелчке пользователя по любому элементу внутри тега 'body' */
```

Хотя этот способ всё ещё функционален, он устарел и не может быть использован для присоединения событий к элементам страницы. Его следует избегать по той же причине, по которой мы избегаем использования атрибутов стилей в разметке для применения стилей к отдельным элементам. В данном случае это размывает границу между семантическим и поведенческим уровнями наших страниц и может быстро привести к проблемам в обслуживании.

Как метод

Это ещё один несколько устаревший подход к присоединённым событиям, хотя он и позволяет строго придерживаться наших скриптов. Мы можем прикреплять функции с помощью встроенных в JavaScript вспомогательных функций:

```
window.onclick = myFunction; /* myFunction будет запущена при щелчке пользователя в окне браузера */
```

Мы также можем использовать анонимную функцию вместо предопределённой:

```
window.onclick = function() {
/* Любой код, размещённый здесь, будет запущен при щелчке
пользователя в окне браузера */
};
```

Этот подход обладает как простотой, так и лёгкостью поддержки, но имеет довольно существенный недостаток: с помощью этого метода мы можем привязать только одно событие за раз.

```
window.onclick = myFunction;
window.onclick = myOtherFunction;
```

В только что приведённом примере вторая привязка перезаписывает первую, поэтому при щелчке пользователя в окне браузера будет запущена только myOtherFunction. Ссылка на myFunction отбрасывается.

addEventListener

Хотя на первый взгляд этот подход немного сложнее, он позволяет нам сохранять логику в скриптах и выполнять несколько привязок к одному объекту. Синтаксис немного более многословен. Мы начинаем с вызова метода addEventListener() целевого объекта, а затем указываем нужное событие и функцию, которую необходимо выполнить, в виде двух аргументов:

```
window.addEventListener("click", myFunction);
```

Обратите внимание, что при использовании этого синтаксиса мы опускаем предыдущее «on» в обработчике событий.

Как и предыдущий метод, addEventListener() можно использовать с анонимной функцией:

```
window.addEventListener("click", function(e) { });
```